

## 챕터 3. 예외 처리와 DTO

게시판 기본 기능을 완성한 오픈이는 다음 날 뿌듯한 마음으로 선배에게 결과물을 보여주었습니다. 선배는 게시글 목록과 상세 조회 기능을 차례로 확인하더니, 이번에는 게시글 번호를 999로 바꿔 다시 조회했습니다. 그러자 상태 코드는 여전히 200(성공)을 반환했지만, 정작 응답에 담긴 게시글 데이터는 null이었습니다.

데이터베이스에 없는 게시글인데 오류가 나지 않고 왜 성공으로 응답하지?

**선배** : "데이터베이스에서 조회된 게시글이 없으면 null 값을 응답하는데, 이 null 값을 서버에서 처리하지 않고 응답하니까 사용자는 정상적인 응답을 받게 될 거예요. 만약 이 상태에서 오류가 발생하면, 사용자는 자신의 문제인지 서버의 문제인지 알 수 없는 오류를 보게 되겠죠. 그래서 서버에서는 이런 **예외 (Exception)**가 발생할 경우에 대비해 미리 예외 처리를 해야 해요. 그래야 어떤 사유로 요청이 실패했는지 클라이언트에게 명확히 알려줄 수 있어요."

선배가 말을 이어갔습니다.

**선배** : "그리고 지금은 서버가 객체를 그대로 응답하는 구조라서 사용자에게 불필요한 정보까지 전달되고, 보안에도 취약해요. 그래서 **DTO**라는 데이터 전송을 위한 객체를 만들고, 이 DTO 타입으로 요청을 받고 응답해야 해요."

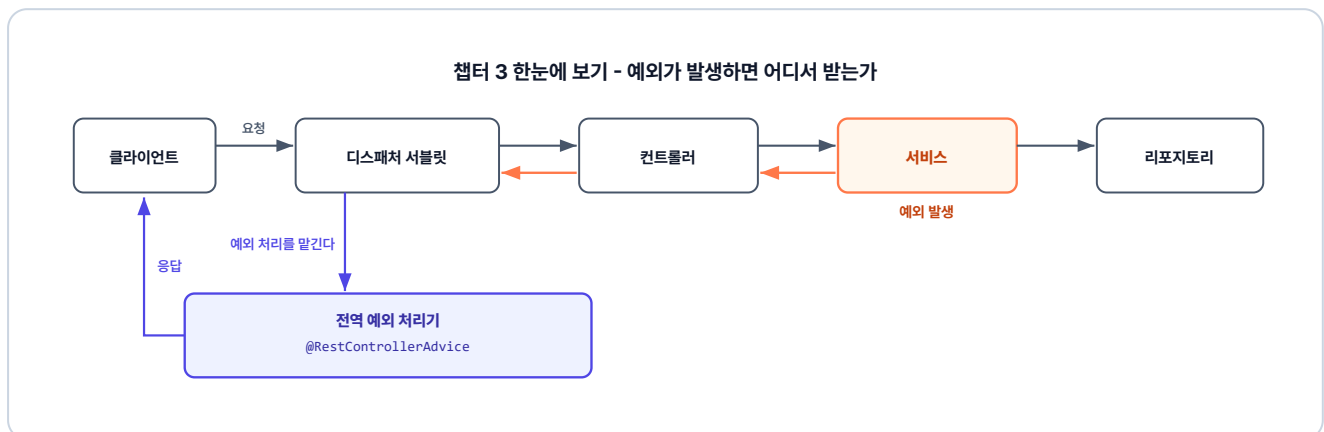


그림 3-1. 챕터 3의 실습 흐름

### 준비하기

## 1. 소스 코드 클론

### 터미널 레포 클론

```
git clone https://github.com/metacoding-06-springboot-v1/start.git
cd start/ch03
```

완성 코드는 final 레포( <https://github.com/metacoding-06-springboot-v1/final> )의 ch03 폴더에서 확인할 수 있습니다.

## 2. 파일 구조

이번 챕터에서 새로 만들거나 고치는 파일만 표시합니다. 나머지는 챕터 2 그대로입니다.

### ch03 디렉토리

```
start/ch03/src/main/java/com/metacoding/spring/
├── board/
│   ├── Board.java                # [참고] 게시물 엔티티
│   ├── BoardController.java      # [작성] 게시물 컨트롤러
│   ├── BoardRequest.java         # [작성] 게시물 요청 DTO
│   ├── BoardRepository.java      # [작성] 게시물 리포지토리
│   ├── BoardResponse.java        # [작성] 게시물 응답 DTO
│   └── BoardService.java         # [작성] 게시물 서비스
├── core/handler/
│   ├── ex/Exception400.java       # [참고] 커스텀 예외
│   ├── ex/Exception401.java       # [참고] 커스텀 예외
│   ├── ex/Exception403.java       # [참고] 커스텀 예외
│   ├── ex/Exception404.java       # [참고] 커스텀 예외
│   ├── ex/Exception500.java       # [참고] 커스텀 예외
│   └── GlobalExceptionHandler.java # [작성] 전역 예외 처리기
```

## 3.1 요청 DTO

챕터 2의 컨트롤러는 요청을 **Board** 엔티티로 직접 받고, 응답할 때도 엔티티를 그대로 반환합니다. 하지만 실무에서는 엔티티를 요청이나 응답에 직접 사용하지 않습니다. 엔티티는 데이터베이스의 테이블 구조와 동일해서, 그대로 사용할 경우 응답에 불필요한 데이터가 노출되거나 테이블 구조가 변경될 때 예상치 못한

오류가 발생할 수 있기 때문입니다. 따라서 클라이언트와 주고받을 때는 필요한 데이터만 담아서 전달할 전용 객체를 사용해야 합니다.

### 3.1.1 DTO

계층 사이에서 데이터 전달만을 목적으로 하는 객체를 **DTO(Data Transfer Object)** 라고 합니다. 컨트롤러와 서비스처럼 서로 다른 계층 사이에서 필요한 값만 담아 전달합니다. DTO를 사용하면 요청으로 받을 값과 응답으로 보낼 값을 필요한 형태로 정할 수 있습니다.

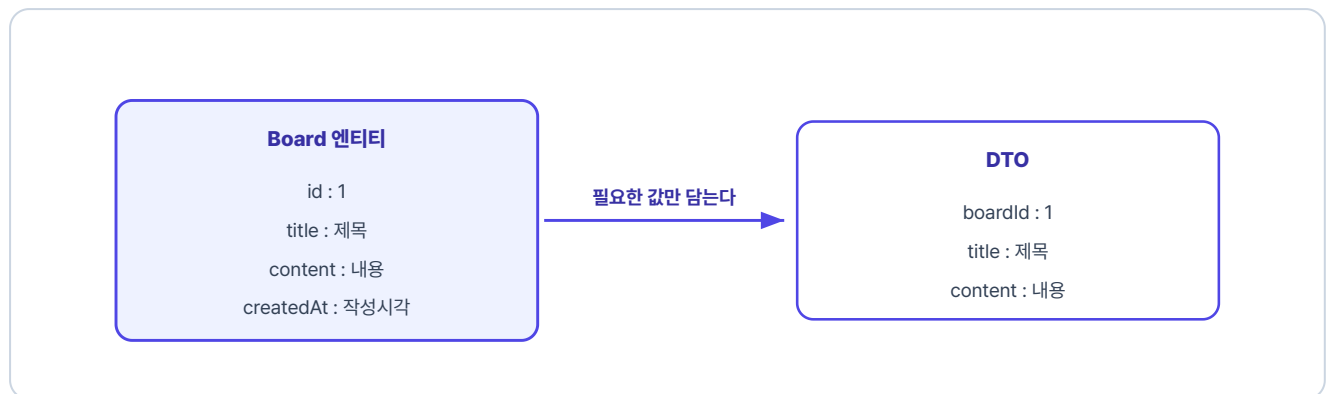


그림 3-2. 엔티티와 DTO가 담는 값

### 3.1.2 엔티티 생성자

데이터베이스에 저장되는 대상은 영속성 컨텍스트가 관리하는 엔티티입니다. 따라서 요청으로 받은 DTO를 데이터베이스에 저장하려면 **Board** 클래스의 생성자로 엔티티를 생성해야 합니다.

**참고** board/Board.java. 빌더로 생성할 수 있게

```
@NoArgsConstructor // 기본 생성자 추가
@Data
@Entity
@Table(name = "board_tb")
public class Board {
    // id, title, content, createdAt 필드 (챕터 2와 같음)

    @Builder // 객체 생성 용도
    public Board(Integer id, String title, String content, LocalDateTime createdAt) {
        this.id = id;
        this.title = title;
        this.content = content;
        this.createdAt = createdAt;
    }
}
```

클래스에 생성자를 새로 정의하면 자바가 자동으로 제공하던 기본 생성자가 사라집니다. 하지만 JPA 엔티티는 기본 생성자가 반드시 있어야 하므로, **@NoArgsConstructor** 어노테이션을 함께 붙입니다.

#### 빌더 패턴(Builder Pattern)이란?

객체를 생성할 때 매개변수 순서에 의존하지 않고, 어떤 필드에 어떤 값이 들어가는지 명시적으로 지정하는 패턴입니다. **@Builder**를 사용하면 복잡한 매개변수 순서를 외울 필요 없이, 원하는 필드에 값을 채워 넣을 수 있습니다.

### 3.1.3 요청 DTO 만들기

요청 DTO는 클라이언트가 전송한 데이터를 담은 클래스입니다. 클라이언트가 데이터를 보내면, 스프링이 요청 바디에 담긴 값들을 이름이 일치하는 DTO 필드에 자동으로 채워줍니다. 덕분에 컨트롤러는 복잡한 변환 과정 없이 이 DTO를 매개변수로 받아 바로 사용할 수 있습니다.

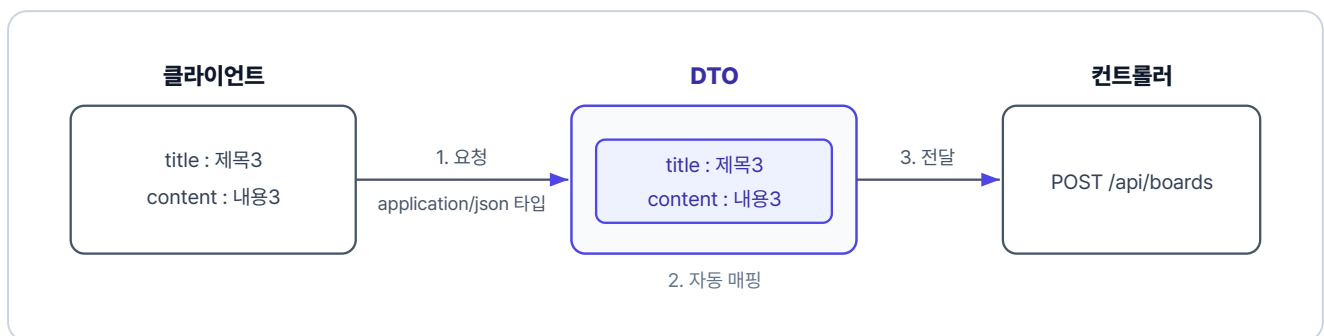


그림 3-3. 요청 데이터와 DTO

다만 게시글 추가 요청을 받는 **SaveDTO**는 값만 담은 일반 객체라서, 그대로는 데이터베이스에 저장할 수 없습니다. 그래서 DTO 안에 엔티티로 바꾸는 **toEntity()** 를 추가합니다.

`board/BoardRequest.java` 를 열어 아래와 같이 작성합니다.

#### 실습 1 board/BoardRequest.java. 요청 DTO와 엔티티 변환

```
public class BoardRequest {
    public record SaveDTO(String title, String content) {

        // DTO를 엔티티로 변환한다
        public Board toEntity() {
            return Board.builder()
                .title(title)
                .content(content)
                .build();
        }
    }
}
```

```

    }
}

public record UpdateDTO(String title, String content) {
}
}

```

`record` 는 데이터 전달을 목적으로 하는 특수 클래스입니다. 필드만 선언하면 생성자와 getter 같은 메서드를 자동으로 만들어 줍니다.

## 3.2 응답 DTO

응답 DTO는 클라이언트에게 내보낼 값만 담는 클래스입니다. 서비스가 조회한 엔티티를 그대로 반환하지 않고, 이 클래스로 옮겨 담아 반환합니다.

`board/BoardResponse.java` 를 열어 아래와 같이 작성합니다.

### 실습 2 board/BoardResponse.java. 응답 DTO

```

public class BoardResponse {

    // 1. 공통 DTO. 엔티티를 받아 보여줄 값만 담는다
    public record DTO(Integer boardId, String title, String content) {

        public DTO(Board board) {
            this(board.getId(), board.getTitle(), board.getContent());
        }
    }

    // 2. 상세용 DTO. 지금은 공통 DTO와 같지만 앞으로 달라진다
    public record DetailDTO(Integer boardId, String title, String content) {

        public DetailDTO(Board board) {
            this(board.getId(), board.getTitle(), board.getContent());
        }
    }
}

```

응답 DTO는 기능마다 따로 만드는 것이 좋습니다. 다만 이 책에서는 편의를 위해 공통 DTO를 사용하고, 게시글 상세만 별도의 DTO를 만듭니다.

### 3.3 JpaRepository 적용

챕터 2에서는 리포지토리에 **EntityManager**를 주입받아 저장과 조회, 삭제 메서드를 직접 작성했습니다. 이 기본 동작들은 **JpaRepository**에 이미 들어 있어, 이 인터페이스를 상속하면 스프링이 대신 구현해 줍니다. 메서드를 작성하지 않아도 저장과 조회, 삭제 메서드를 호출할 수 있습니다.

`board/BoardRepository.java`를 열어 아래와 같이 변경합니다.

**실습 3** `board/BoardRepository.java`. **JpaRepository** 상속

```
public interface BoardRepository extends JpaRepository<Board, Integer> {  
}
```

리포지토리는 클래스가 아니라 인터페이스로 선언합니다. `JpaRepository`를 상속할 때 다룰 엔티티 타입과 기본 키 타입을 차례로 적습니다.

리포지토리의 조회 메서드는 반환 타입을 **Optional**로 선언할 수 있습니다. **Optional**은 존재할 수도, 존재하지 않을 수도 있는 값을 감싸는 래퍼(**Wrapper**) 클래스로, 주로 null로 인한 에러를 방지하기 위해 사용됩니다. 이렇게 선언하면 값이 없을 때 null 대신 빈 **Optional**이 반환되므로, 게시글을 찾지 못한 경우를 예외로 처리할 수 있습니다.

자주 쓰는 메서드는 다음과 같습니다.

| 메서드                         | 하는 일                                      |
|-----------------------------|-------------------------------------------|
| <code>save(엔티티)</code>      | 저장하고 저장된 엔티티를 반환합니다                       |
| <code>findById(기본 키)</code> | 기본 키로 한 건을 조회해 <b>Optional</b> 에 담아 반환합니다 |
| <code>findAll()</code>      | 전체를 <b>List</b> 로 반환합니다                   |
| <code>delete(엔티티)</code>    | 삭제합니다                                     |

### 3.4 예외 처리 추가

스프링에서 예외가 어떻게 처리되는지 흐름부터 알아보겠습니다.

### 3.4.1 예외 처리 흐름

스프링에서 발생한 예외는 요청이 들어왔던 길을 되돌아가 디스패처 서블릿까지 전달됩니다. 이때 디스패처 서블릿은 스프링 내부에 예외를 오류 응답으로 바꿀 곳이 있는지 찾고, 찾지 못하면 톰캣으로 예외를 넘깁니다.

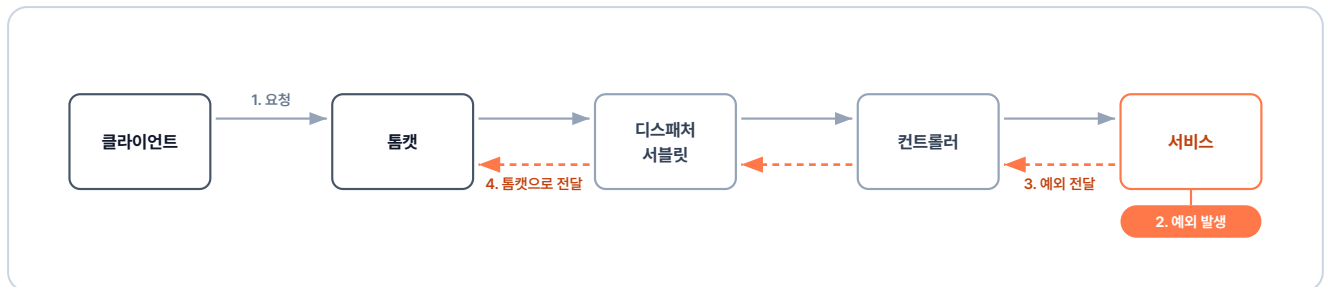


그림 3-4. 예외가 톰캣까지 전달되는 흐름

예외를 넘겨받은 톰캣은 오류 응답을 만들기 위해 스프링 부트에 설정된 `/error` 주소로 요청을 다시 보냅니다. 그러면 스프링 부트의 **기본 오류 컨트롤러(BasicErrorController)** 가 이를 받아 기본 오류 정보를 담아 응답합니다.

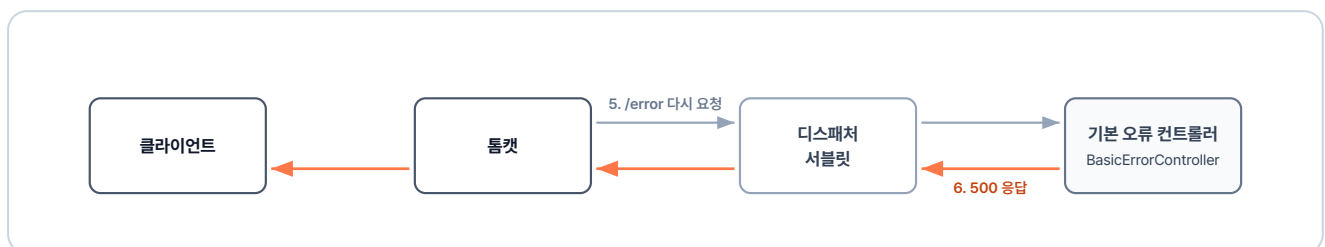


그림 3-5. /error 재요청과 오류 응답

#### 기본 오류 컨트롤러(BasicErrorController)란?

스프링 부트가 기본으로 제공하는 오류 처리 컨트롤러로, `/error` 주소를 담당합니다. 웹 브라우저의 요청에는 기본 오류 화면(HTML)을, API 요청에는 발생 시각, 상태 코드, 요청 주소 같은 기본 오류 정보를 담은 JSON을 반환합니다. 상태 코드에는 톰캣이 전달한 값이 그대로 들어가므로, 처리되지 않은 예외라면 500, 주소에 맞는 컨트롤러가 없는 요청이라면 404가 됩니다.

### 3.4.2 전역 예외 처리기

이 기본 흐름에서는 예외 발생 시 반환할 응답을 서버가 직접 제어하기 어렵습니다. 기본 오류 컨트롤러가 정해진 형식으로만 응답을 생성하므로, 게시글을 찾지 못한 경우에도 상태 코드 500이 반환되며 정확한 실패 사유도 담기지 않기 때문입니다.

그래서 예외가 톰캣까지 넘어가기 전에 처리될 수 있도록, 스프링 내부에 **전역 예외 처리기(Global Exception Handler)** 를 추가합니다. 전역 예외 처리기는 디스패처 서블릿으로 전달된 예외를 한곳에서 전담하는 클래스로, 발생한 예외를 서버가 지정한 상태 코드와 실패 사유가 담긴 응답으로 바꾸어 줍니다.

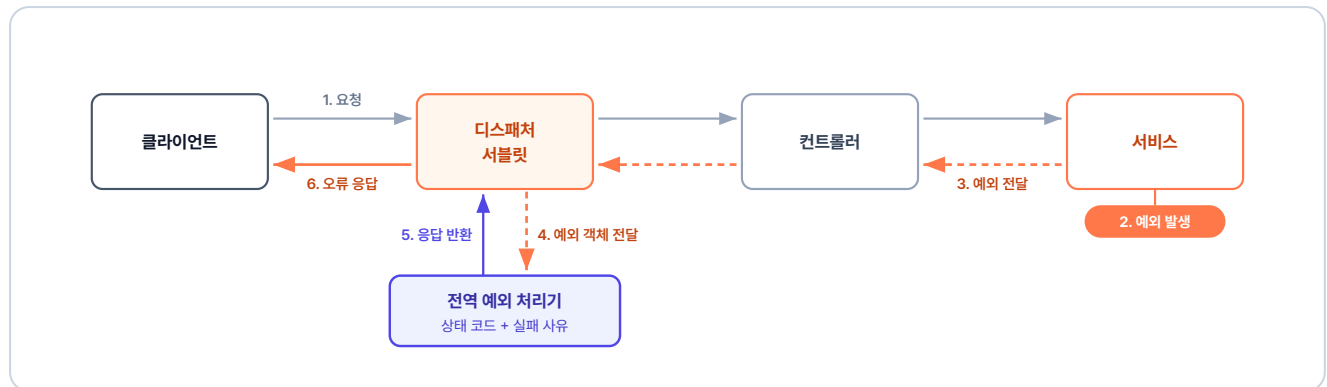


그림 3-6. 전역 예외 처리기가 있을 때의 흐름

### 전역 예외 처리기가 받지 못하는 예외

전역 예외 처리기는 디스패처 서블릿까지 전달된 예외만 처리할 수 있습니다. 디스패처 서블릿에 진입하기 전인 필터 단계에서 발생한 예외는 가로채지 못하고, 앞에서 본 흐름대로 톰캣을 거쳐 기본 오류 컨트롤러가 응답합니다. 따라서 필터에서는 예외 응답을 직접 만들거나, 예외를 발생시키지 않고 요청을 통과시켜 서비스에서 처리하도록 해야 합니다.

### 3.4.3 전역 예외 처리기 만들기

이제 앞에서 살펴본 전역 예외 처리기를 직접 만들어 보겠습니다. 클래스에 **@RestControllerAdvice**를 적용하면, 스프링은 해당 클래스를 전역 예외 처리기로 인식합니다. 이 클래스는 모든 컨트롤러에서 발생하는 예외를 한곳에서 모아 처리하고 JSON으로 응답합니다.

`core/handler/GlobalExceptionHandler.java` 를 열어 아래와 같이 작성합니다.

#### 실습 4 core/handler/GlobalExceptionHandler.java. 전역 예외 처리

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    // 1. 커스텀 예외는 저마다의 상태 코드로 바꾼다
    @ExceptionHandler(Exception400.class)
    public ResponseEntity<?> exApi400(Exception400 e) {
        return Resp.fail(HttpStatus.BAD_REQUEST, e.getMessage());
    }
}
  
```

```

@ExceptionHandler(Exception401.class)
public ResponseEntity<?> exApi401(Exception401 e) {
    return Resp.fail(HttpStatus.UNAUTHORIZED, e.getMessage());
}

@ExceptionHandler(Exception403.class)
public ResponseEntity<?> exApi403(Exception403 e) {
    return Resp.fail(HttpStatus.FORBIDDEN, e.getMessage());
}

@ExceptionHandler(Exception404.class)
public ResponseEntity<?> exApi404(Exception404 e) {
    return Resp.fail(HttpStatus.NOT_FOUND, e.getMessage());
}

@ExceptionHandler(Exception500.class)
public ResponseEntity<?> exApi500(Exception500 e) {
    return Resp.fail(HttpStatus.INTERNAL_SERVER_ERROR, e.getMessage());
}

// 2. 나머지 모든 예외는 500으로 처리한다
@ExceptionHandler(Exception.class)
public ResponseEntity<?> exUnknown(Exception e) {
    return Resp.fail(HttpStatus.INTERNAL_SERVER_ERROR, "서버 오류가 발생했습니다");
}
}

```

전역 예외 처리기는 전달받은 예외를 타입별로 구분하여 처리합니다. 각각의 예외 상황에 맞춰 미리 지정해 둔 상태 코드와 구체적인 실패 사유를 응답에 담아냅니다. 이때 별도로 지정해 두지 않은 예상치 못한 예외들은 모두 상태 코드 500으로 일괄 처리합니다.

### 3.4.4 커스텀 예외 만들기

전역 예외 처리기가 상황에 맞게 응답하려면, 예외가 어떤 상황에서 발생했는지 구분할 수 있어야 합니다. 기본으로 제공되는 예외만으로는 이를 세밀하게 구별하기 어렵기 때문에, 개발자가 특정 상황에 맞는 예외를 만들어 사용하는데 이를 **커스텀 예외(Custom Exception)** 라고 부릅니다.

서비스에서 커스텀 예외가 발생하면, 이 예외 객체는 디스패처 서블릿을 거쳐 전역 예외 처리기로 전달됩니다. 전역 예외 처리기는 이를 상태 코드와 실패 사유를 담은 응답으로 바꿔 클라이언트에게 반환합니다.

예를 들어 **Exception404**는 게시글처럼 찾는 자원이 없을 때 발생시킬 예외 클래스입니다.

참고

core/handler/ex/Exception404.java. 커스텀 예외

```
public class Exception404 extends RuntimeException {  
    public Exception404(String message) {  
        super(message);  
    }  
}
```

**RuntimeException**을 상속받아 구현하면, 예외를 발생시키는 메서드와 그 메서드를 호출하는 메서드 모두 `try-catch` 나 `throws` 같은 예외 처리 코드를 작성하지 않아도 됩니다. 같은 경로에 있는 **Exception400**, **Exception401**, **Exception403**, **Exception500** 클래스 역시 이름만 다르고 구조는 동일합니다.

이 책에서 사용하는 상태 코드는 다음과 같습니다.

| 상태 코드 | 뜻                   | 예                       |
|-------|---------------------|-------------------------|
| 400   | 서버가 정한 규칙에 맞지 않는 요청 | 제목 없이 게시물 작성            |
| 401   | 인증하지 않은 요청          | 로그인하지 않고 게시물 추가         |
| 403   | 인증했지만 권한이 없는 요청     | 다른 사람이 쓴 게시물 수정         |
| 404   | 찾을 수 없는 자원          | 없는 게시물 조회               |
| 500   | 서버 내부 오류            | 앞의 네 가지에 해당하지 않는 나머지 예외 |

## 3.5 서비스와 컨트롤러에 적용

### 3.5.1 서비스

이제 서비스에서 예외 처리를 해보겠습니다.

**BoardService**가 DTO로 값을 주고받고, 게시글이 없으면 예외를 발생시키도록 변경합니다.

`board/BoardService.java`를 열어 아래와 같이 작성합니다.

실습 5

board/BoardService.java. 주고받는 타입을 DTO로, 없으면 예외

```
public List<BoardResponse.DTO> 게시글목록() {
    return boardRepository.findAll().stream()
        .map(BoardResponse.DTO::new)
        .toList();
}

public BoardResponse.DetailDTO 게시글상세(Integer boardId) {
    Board board = boardRepository.findById(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    return new BoardResponse.DetailDTO(board);
}

@Transactional
public BoardResponse.DTO 게시글추가(BoardRequest.SaveDTO requestDTO) {
    Board savedBoard =
        boardRepository.save(requestDTO.toEntity()); // DTO -> 엔티티
    return new BoardResponse.DTO(savedBoard); // 저장된 게시글 반환
}

@Transactional
public BoardResponse.DTO 게시글수정(Integer boardId,
    BoardRequest.UpdateDTO requestDTO) {
    Board board = boardRepository.findById(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    // 데이터체크
    board.setTitle(requestDTO.title());
    board.setContent(requestDTO.content());
    return new BoardResponse.DTO(board); // 수정된 게시글 반환
}

@Transactional
public void 게시글삭제(Integer boardId) {
    Board board = boardRepository.findById(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    boardRepository.delete(board);
}
```

리포지토리가 반환한 **Optional**에서 값을 꺼낼 때는 `orElseThrow()` 를 사용합니다. 이 메서드는 값이 있으면 그 값을 반환하고, 비어 있으면 인자로 전달한 예외를 발생시킵니다.

### 3.5.2 컨트롤러

컨트롤러도 엔티티 대신 DTO로 값을 주고받도록 변경합니다.

board/BoardController.java 를 열어 아래와 같이 작성합니다.

**실습 6** board/BoardController.java. 요청·응답 타입을 DTO로

```
@GetMapping
public ResponseEntity<?> findAll() {
    List<BoardResponse.DTO> respDTOList = boardService.게시글목록();
    return Resp.ok(respDTOList);
}

@GetMapping("/{boardId}")
public ResponseEntity<?> findById(@PathVariable("boardId") Integer boardId) {
    BoardResponse.DetailDTO respDTO = boardService.게시글상세(boardId);
    return Resp.ok(respDTO);
}

@PostMapping
public ResponseEntity<?> save(@RequestBody BoardRequest.SaveDTO requestDTO) {
    BoardResponse.DTO respDTO = boardService.게시글추가(requestDTO);
    return Resp.ok(respDTO);
}

@PutMapping("/{boardId}")
public ResponseEntity<?> update(@PathVariable("boardId") Integer boardId,
    @RequestBody BoardRequest.UpdateDTO requestDTO) {
    BoardResponse.DTO respDTO = boardService.게시글수정(boardId, requestDTO);
    return Resp.ok(respDTO);
}

@DeleteMapping("/{boardId}")
public ResponseEntity<?> deleteById(@PathVariable("boardId") Integer boardId) {
    boardService.게시글삭제(boardId);
    return Resp.ok(null);
}
```

이제 프로젝트를 실행해 결과를 확인해 보겠습니다. 먼저 1번 게시글을 조회하면 게시글 데이터가 정상적으로 응답됩니다.

**Hoppscotch** 게시글 상세 조회

GET <http://localhost:8080/api/boards/1>

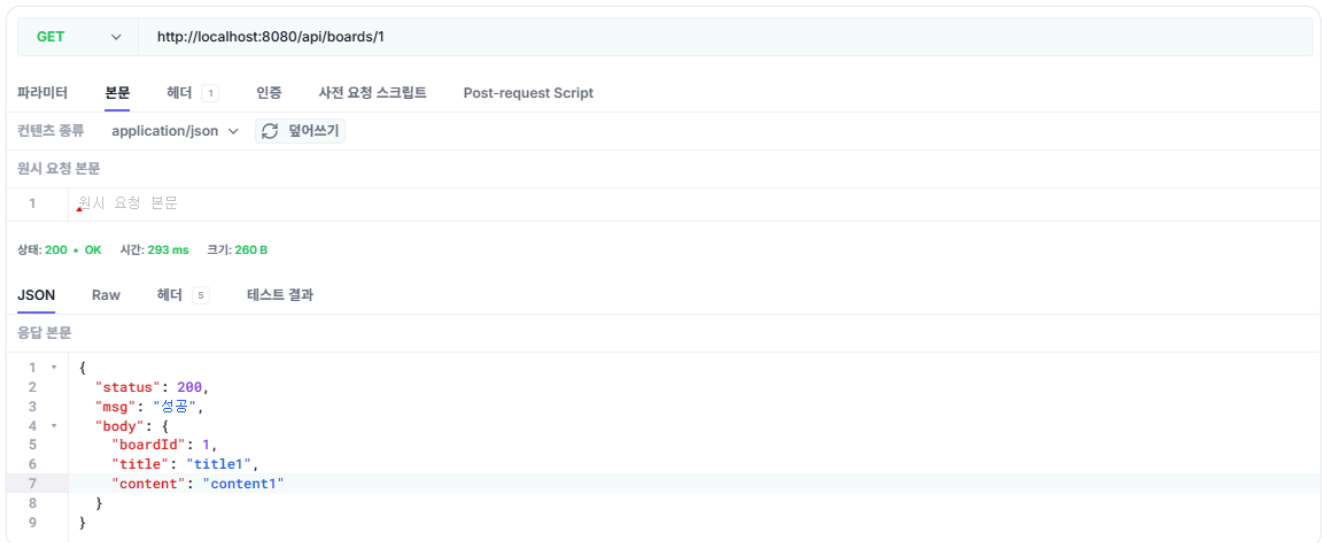


그림 3-7. 상세 조회 응답

다음으로 예외를 발생시켜 보겠습니다. 더미 데이터에 없는 999번 게시글을 조회해 응답 결과를 확인합니다.

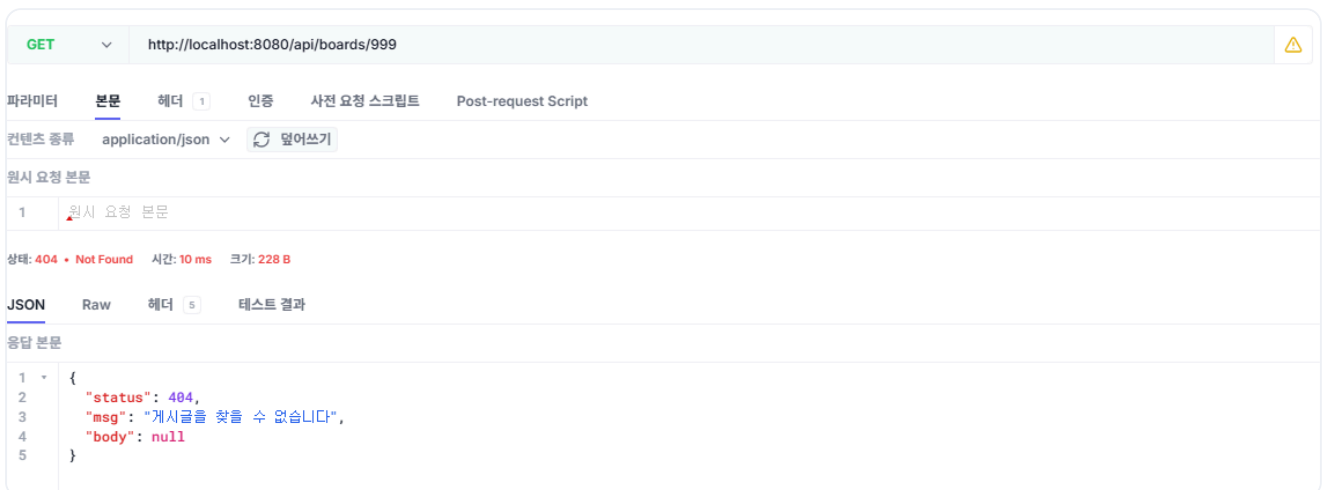
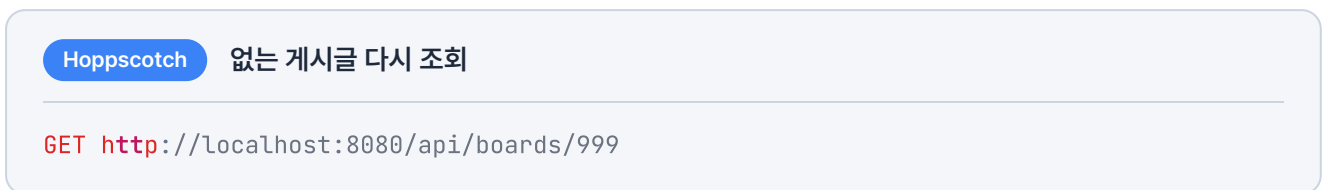


그림 3-8. 404 응답

챕터 2에서는 엔티티를 직접 요청과 응답에 사용했지만, 이번 챕터에서는 DTO를 활용해 데이터를 전달하는 구조로 개선했습니다. 또한 예외 처리를 추가해, 문제가 발생하더라도 클라이언트가 상태 코드와 실패 사유를 응답으로 받을 수 있게 했습니다.

다음 챕터에서는 회원 엔티티를 추가해 게시글에 작성자 정보를 담고, 이를 바탕으로 인증과 인가 기능을 구현해 보겠습니다.

### 이것만은 기억하자

- 엔티티를 요청과 응답에 직접 사용하지 않고 DTO로 주고받습니다. 테이블 구조가 바뀌어도 클라이언트와 주고받는 형식은 영향을 받지 않습니다.
- 요청이 실패하면 상태 코드와 사유를 담아 응답합니다. 서비스가 예외를 발생시키면 전역 예외 처리기가 한 곳에서 받아 상태 코드에 맞는 JSON으로 바꿉니다.